

Hardware Root of Trust in RISC-V Systems: Analysis and Evaluation of Open Source Approaches

Pau Romaguera Palomé

November 15, 2025

Resum– L'evolució ràpida del hardware d'altres prestacions ha transformat la indústria dels semiconductors, possibilitant avenços en intel·ligència artificial (IA) i computació d'altres prestacions (HPC). A mesura que aquests sistemes esdevenen més sofisticats, garantir la seguretat del xip és essencial per protegir les dades que processen. Aquesta tesi avalua funcionalitats de seguretat de codi obert en sistemes basats en RISC-V mitjançant proves adversàries, centrant-nos en mecanismes de protecció de memòria com la protecció física de memòria (PMP) i la PMP millorada (ePMP), així com en la verificació criptogràfica dins del Secure Boot. Prenent com a cas d'estudi el System on Chip (SoC) anomenat Earlgrey d'OpenTitan [12] [2] [3], afeblim o eliminem proteccions seleccionades per quantificar la contribució de cada mecanisme i derivar recomanacions pràctiques per enfortir dissenys de Hardware Root of Trust (HROt).

Paraules clau– Computació d'altres prestacions (HPC), plataformes RISC-V, intel·ligència artificial (IA), sistemes arrelats en hardware (HROt), Protecció Física de Memòria (PMP)

Abstract– The rapid evolution of high-performance hardware has transformed the semiconductor industry, enabling advances in artificial intelligence (AI) and high-performance computing (HPC). As these systems grow more sophisticated, ensuring chip security is essential to protect the data they process. This thesis evaluates open-source security features in RISC-V-based systems using adversarial testing, focusing on memory protection mechanisms such as Physical Memory Protection (PMP) and enhanced PMP (ePMP) and cryptographic verification in secure boot. Using OpenTitan's [12] [2] [3] Earlgrey System on Chip (SoC) as the case study, we weaken or remove selected protections to quantify each mechanism's contribution and derive practical recommendations for strengthening hardware root of trust (HROt) designs.

Keywords– High Performance Computing (HPC), RISC-V platforms, artificial intelligence (AI), System on Chip (SoC), Hardware Root of Trust (HROt), Physical Memory Protection (PMP)



1 INTRODUCTION - CONTEXT OF THE THESIS

MODERN semiconductor designs increasingly embed security features directly in hardware so that sensitive operations can be protected without performance penalties.

- E-mail de contacte: pauromaguera1@gmail.com
- Menció: Enginyeria de Computadors
- Treball tutoritzat per: Marta Baldrís Calmet (Openchip and Software Technologies S.L.) Raimon Casanova Mohr (Departament de Microelectrònica i Sistemes Electrònics)
- Curs 2025/26

The open RISC-V instruction set architecture (ISA) [6] accelerates this shift by enabling community-driven security extensions and cryptographic instructions that, together with dedicated accelerators and on-chip hardware roots of trust (HROt), provide robust security with low overhead, safeguarding data and code integrity for end users.

In this thesis, we focus on the HROt model in RISC-V systems, using OpenTitan as the primary case study [12]. Our analysis centers on the mechanisms that underpin a secure boot chain, particularly Physical Memory Protection (PMP) [8][9] and its enhanced variant ePMP [11], and lastly, cryptographic verification of early firmware.

These mechanisms share common principles and integrate complementary defenses, such as region-based memory isolation (PMP/ePMP), cryptographic firmware verifi-

cation (Secure Boot) [10], and strict separation of privileges across boot stages.

HRoT functionality is typically integrated within a System-on-Chip (SoC) and may include a dedicated core alongside hardened peripherals and memories to minimize the attack surface and resist software-based compromise. In this context, a clear separation of roles across boot stages, each with narrowly scoped privileges, enables more robust and reliable protection.

2 OBJECTIVES

The main objective of this thesis is to analyze and empirically prove the level of security of the open-source EarlGrey SoC from OpenTitan. In particular, the contribution of certain security mechanisms will be examined, such as Secure Boot and enhanced Physical Memory Protection (ePMP) [11], to verify how each function contributes to the system's overall security.

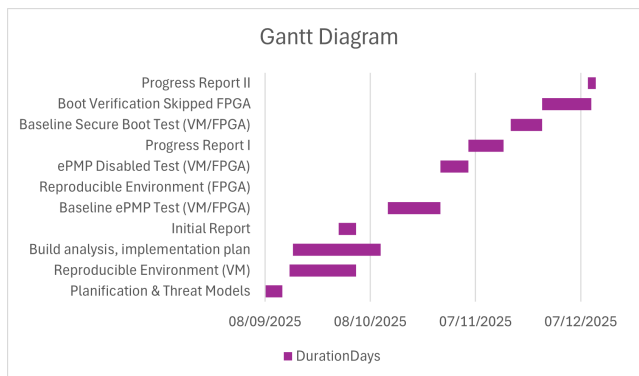
To stress-test these mechanisms, we will use ablation tests. This involves intentionally modifying or removing specific security policies or functions in order to assess how vulnerable the SoC becomes to certain attack vectors.

In doing so, we can identify which measures are most critical for protecting the system and establish a hierarchy of defenses based on their actual effectiveness.

3 METHODOLOGY & PLANIFICATION

The project has been organized according to the Scrum methodology. The duration of tasks are planned in two-week iterative cycles, called sprints. Each task is expected to be completed within a sprint or, when necessary, decomposed into smaller sub-tasks to fit the two-week cycle. In addition, daily standup meetings are held with Openchip's "Security Features" team to share progress and issues, Sprint Reviews are conducted to evaluate results, and retrospectives identify opportunities for improvement.

As mentioned earlier, each task has been structured to fit biweekly cadence, although some larger tasks have spanned two sprints, depending on issues encountered during execution. The timeline can be seen in the initial Gantt chart in Fig. 1, which shows the sequence and duration of the main activities.



5.2 In-scope threats

The following threats are considered within scope for this work:

1. **Secure Boot bypass:** attempts to execute unsigned or tampered firmware images or skip verification steps.
2. **Memory protection violations:** accessing or executing from regions that should be protected (for example via ePMP) or modifying configuration parameters or secrets that should be inaccessible.
3. **Rollback/downgrade attacks:** replaying older firmware or data images, or circumventing anti-rollback counters to reduce security level.

5.3 Out-of-scope threats

Certain threat vectors are outside the scope of this thesis:

1. **Hardware attacks:** attempts to break the chip via decapsulating and micro-probing internal interconnects, are not considered for this thesis.
2. **High-layer software vulnerabilities:** vulnerable applications running on top of the bootloader/firmware once the secure boundary is established. Bad coding practices shall not be considered, for example, buffer overflows.
3. **Remote compromises:** any kind of cloud infrastructure, host PC or remote development compromises, meaning attacks outside the device itself, shall not be considered either.

Therefore, the chosen threat model aims to balance realistic deployment risks with feasible experimental evaluation. The assumed adversary, one with physical but non-invasive access to the device, reflects a plausible scenario in which a malicious individual acquires hardware, modifies its firmware and resells it as new. The in-scope threats directly relate to the security mechanisms evaluated in this thesis, namely the Secure Boot process and its enforcement of memory protection, and cryptographic verification. Excluding extreme invasive attacks acknowledges the limitations of our experimental setup, while still enabling meaningful insights into system behaviour under misconfigurations and abnormal conditions.

6 SYSTEM UNDER TEST: SECURE BOOT

OpenTitan’s secure boot chain ensures that every piece of firmware executed on the device is both authentic (signed by an approved key) and untampered (the signed hash matches with the firmware hash). The booting sequence is as follows:

6.1 OpenTitan Logical Entities

OpenTitan distinguishes between two principal logical entities involved in device provisioning, key management, and firmware authenticity: the Silicon Creator and the Silicon Owner. In this approach, OpenTitan acknowledges that there might be overlap within these actors depending on the final use case.

1. **Silicon Creator** The Silicon Creator is the entity responsible for manufacturing the device and injecting the initial set of public keys into the chip during production. These keys authenticate the earliest stages of the secure boot chain, specifically the immutable ROM and the ROM_EXT firmware. Because these stages establish the foundational root of trust, only the Silicon Creator is permitted to sign them. The identity of the Silicon Creator is fixed for the lifetime of the device and cannot be altered once manufacturing has completed.
2. **Silicon Owner** The Silicon Owner is the individual or organization that takes ownership of the device after manufacture. Ownership transfer is performed by provisioning a new set of public keys onto the device, enabling the owner to authenticate and sign all subsequent boot stages after ROM_EXT (BL0 and higher level firmware). Unlike the Silicon Creator, the Silicon Owner may change over the device’s lifetime, allowing devices to be securely transferred between organisations or end-users without compromising the established root of trust.

6.2 Boot Stages in OpenTitan

1. **ROM (Stage 0)** The first code executed on an OpenTitan SoC resides in an immutable mask ROM embedded directly in silicon. As this component cannot be updated post-manufacture, it constitutes the hardware root of trust and is responsible for authenticating all subsequent firmware stages. Upon power on, the processor begins execution at the ROM, which establishes a minimal trusted execution environment by configuring the enhanced Physical Memory Protection (ePMP) so that only the ROM region is executable. It then performs essential early initialization, including setting up the C runtime and enabling a small subset of peripherals. The ROM reads a “manifest” structure from flash, which specifies the location of the next boot stage (ROM_EXT), selects the appropriate RSA-3072 public key modulus for verification, and provides a cryptographic signature over the ROM_EXT image together with a set of usage constraints. These usage constraints are not the raw device-specific values themselves, but rather a selection mask indicating which hardware-provided fields (such as lifecycle state or device identifiers) the ROM must incorporate into the hash. This mechanism allows the signer of a ROM_EXT image to restrict its validity to specific device states or device classes. The ROM computes a SHA-256 digest over the selected usage-constraint values concatenated with the ROM_EXT contents and verifies the signature using the public key indicated in the manifest. If verification succeeds, the ROM reconfigures ePMP to allow execution of the ROM_EXT flash region and transfers control. If verification fails, the boot process halts to prevent execution of untrusted code. Halting could either mean attempting another boot or shutting off the system completely.
2. **ROM_EXT (Stage 1)** The ROM_EXT (or “ROM extension”) constitutes the second stage of the secure

boot process and is stored in flash memory. Unlike the immutable ROM, ROM_EXT can be updated by the Silicon Creator, making its integrity critical to the security of the entire boot chain. ROM_EXT mirrors the core responsibility of the ROM: it authenticates the next stage in the boot sequence, known as BL0. To do so, it parses the BL0 manifest, computes a SHA-256 hash over the BL0 image, and verifies the accompanying RSA-3072 signature using the public key specified in the manifest.

Only upon successful verification does ROM_EXT adjust the ePMP configuration to permit execution from the BL0 flash region. Control is then transferred to BL0. As BL0 is owner-controlled firmware, meaning it can be modified by the device owner rather than the Silicon Creator, the ROM_EXT verification step ensures that unauthorized or malicious modifications are detected before execution.

3. **BL0 (Stage 2)** BL0 is the first owner-controlled firmware stage executed after the ROM and ROM_EXT have completed their respective authentication duties. Its primary role is to perform platform-specific initialization, such as configuring bus interfaces, establishing DMA windows, and relocating larger firmware components into on-chip SRAM to improve performance. After preparing the system environment for higher-level software, BL0 continues the secure boot chain by parsing the manifest of the next firmware stage, computing the corresponding SHA-256 digest over its contents, and verifying the RSA-3072 signature using the public key authorized by ROM_EXT. Execution is delegated to the next stage only if this verification succeeds.

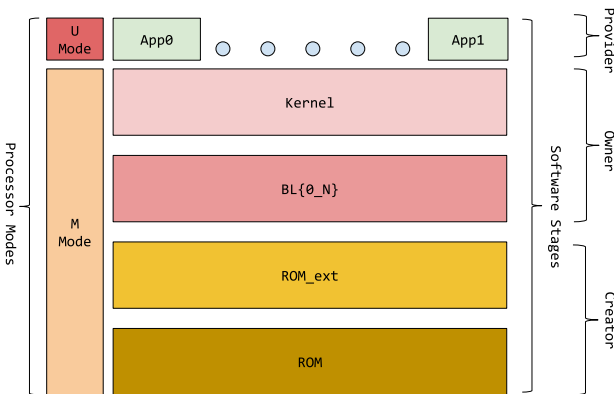


Fig. 2: Secure Boot Diagram

6.3 Memory Protection

OpenTitan’s 32-bit Ibex RISC-V core implements the enhanced Physical Memory Protection extension (ePMP). We first describe the base RISC-V Physical Memory Protection (PMP) in detail and then explain how ePMP strengthens early boot security and machine mode enforcement on this SoC.

6.3.1 PMP

To support secure processing and to contain software faults, it is often necessary to restrict which physical addresses software can access. The Physical Memory Protection (PMP) unit provides per-hart controls - where a hart refers to a “hardware thread” or an independent execution pipeline - that allow machine mode to specify access permissions (read, write, execute) for defined physical memory regions. By configuring these PMP entries, the system can enforce fine-grained isolation and determine which areas of the physical address space are accessible to lower-privileged software, thus improving overall memory safety and robustness.

Different RISC-V cores may implement varying PMP entry configurations. For instance, OpenTitan’s Ibex core uses 16 PMP entries in total, meaning sixteen memory regions to which we can enforce certain access modes, although different implementations differ in rule number. The standard uses two kinds of control status registers (CSR) to implement protection, namely:

6.3.1.1 pmpaddrN registers store the address fields that define the boundaries of PMP enforced memory regions. The number of pmpaddr registers always matches the number of PMP entries implemented by the core. Importantly, the width of these registers is not determined by the CPU’s XLEN (32 or 64 bit). Rather, it is defined by the physical address width supported by the implementation. For example, an RV32 system that implements the Sv32 ?? virtual-memory scheme must support 34-bit physical addresses, so its pmpaddr registers expose the field address[33:2] as a WARL (Write Any, Read Legal) value. In systems without a memory management unit (MMU), such as OpenTitan’s Ibex core, therefore without virtual memory, the physical address space is only 32 bits wide, so the upper address bits simply read as zero and any writes to them are ignored according to WARL semantics.

6.3.1.2 pmpcfgN Each pmpcfg CSR is four bytes wide and encodes four PMP entries (one byte per entry). Concretely:

1. pmpcfg0: holds configurations of pmpaddr0...3
2. pmpcfg1: holds configurations of pmpaddr4...7
3. pmpcfg2: holds configurations of pmpaddr8...11
4. pmpcfg3: holds configurations of pmpaddr12...15

As stated above, each byte in a pmpcfg register encodes the configuration for one PMP entry. This byte specifies the read, write, and execute permissions (R/W/X), selects the address-matching mode via the A field (*OFF*, *TOR*, *NA4*, or *NAPOT*), and includes the lock bit L, which, when set, locks the entry until reset and causes M-mode to enforce the permissions. Bits [6:5] are reserved and must be written as zero. Together with the corresponding pmpaddr[i], this configuration defines the protected region and its allowed accesses.

TABLE 1: ONE-BYTE LAYOUT OF A PMPCFG ENTRY

Bits	Field	Allowed	Meaning / effect
7	L	0, 1	1: lock entry and enforce permissions on M-mode; 0: not locked (base PMP enforces on S/U only).
6–5	(reserved)	must be 0	Reserved; writes should be zero (WARL).
4–3	A[1:0]	00, 01, 10, 11	Address-matching mode.
2	X	0, 1	Execute permission for the matched region.
1	W	0, 1	Write permission for the matched region.*
0	R	0, 1	Read permission for the matched region.

* The combination R=0 and W=1 is reserved in PMP; implementations treat it as disallowing writes (i.e., equivalent to R=0, W=0).

6.3.1.3 PMP address-matching modes (A field) These are the address-matching modes defined by RISC-V PMP. Each mode trades off granularity versus the number of PMP entries consumed to describe a region.

1. **00: OFF** The entry is disabled and defines no region. Its permissions are ignored. OFF entries can still serve as the lower bound for a following TOR region (via their `pmpaddr` value).
2. **01: TOR (Top of Range)** Defines a half-open interval $[lower, upper)$. The *upper* bound is taken from the current entry’s `pmpaddr[i]`, while the *lower* bound is the previous entry’s `pmpaddr[i-1]` (or zero for $i=0$). TOR is convenient when the final size is only known after linking (e.g., the end of a program image). Note that TOR typically consumes *two* entries (one to hold the lower bound, one to hold the upper bound and permissions).
3. **10: NA4 (Naturally Aligned 4 bytes)** Protects a single 4-byte word at the address given by `pmpaddr[i]`. Useful to cover very small holes such as a stack guard word or a narrow MMIO register. It consumes a single entry but offers minimal coverage.
4. **11: NAPOT (Naturally Aligned Power of Two)** Covers a memory space that is aligned to a size of a power of two with just a single entry. This is ideal when the block size is known and aligned, for instance, a 16 KiB ROM or a 64 KiB flash partition.

6.3.1.4 The mcause CSR is a Machine-mode control and status register (CSR) that records the cause of the most recent trap taken into M-mode. A trap is either an exception (illegal instruction or a protection fault), or an interrupt (like timers or external event). In the context of this work, PMP is the primary source of protection-fault exceptions: attempting to execute from a non-X region raises an *instruction access fault*, reading from a non-R region raises a *load access fault*, and writing to a non-W region raises a *store access fault*. When such a trap occurs, the core writes `mcause` with two pieces of information: a flag indicating

whether the trap is an interrupt, and a cause code identifying the specific reason. The handler uses `mcause` together with `mepc`, which holds the program counter at which the trap was taken, to determine the appropriate service routine and, if necessary, to adjust control flow by updating `mepc` to the following instruction to be executed. If `mepc` is not updated and the underlying condition is not corrected, the processor resumes at the same faulting instruction and immediately retriggers the exception, yielding a trap loop with no forward progress. The register width follows the implementation’s XLEN, but for practical purposes, it is sufficient to note that the most significant bit of `mcause` distinguishes interrupts from exceptions and the remaining bits carry the cause code.

6.3.1.5 Representative exception causes (Ibex) In the context of PMP and secure boot evaluation, the following Ibex exception codes are the most relevant. The names and numeric values correspond to the `ibex_exc_t` enumeration used by the firmware.

1. **kIbexExcInstrAccessFault (1)** Raised when an instruction fetch targets an address without execute permission, for example, PMP denies X or otherwise cannot be accessed.
2. **kIbexExcIllegalInstrFault (2)** Raised when the fetched instruction encoding does not correspond to any valid opcode for the implemented ISA or is not permitted in the current context. In this work, it is common to execute from regions prefilled with all ones (for example, `0xFFFF...`). If execute permission is denied by PMP/ePMP, the trap observed is an *instruction access fault*; if execution is permitted, the all-ones word does not decode to a legal instruction and therefore raises an *illegal instruction fault*.
3. **kIbexExcLoadAccessFault (5)** Raised when a load violates access policy, for example, PMP denies R or targets an inaccessible region.
4. **kIbexExcStoreAccessFault (7)** Raised when a store/AMO violates access policy, for example, PMP denies W or targets an inaccessible region.
5. **kIbexExcMax (31)** Defines the maximum code value in the `ibex_exc_t` enumeration. It is not generated by hardware as a trap cause, rather it is typically used in firmware as a special “no exception recorded” marker.

Therefore, the `mcause` register provides a clear means of identifying which exception was raised by a given instruction fetch or data access during our experiments with PMP-configured regions. `kIbexExcInstrAccessFault` distinguishes *execute* permission failures from mere decode errors, that are captured by `kIbexExcIllegalInstrFault` when execution is permitted but the bits do not form a valid opcode, while `kIbexExcLoadAccessFault` and `kIbexExcStoreAccessFault` reveal *read* and *write* violations, respectively. In practice, this quartet lets us diagnose any kind of misconfigurations or find unexpected

vulnerabilities. The `kIbexExcMax` value is not produced by hardware; it is used in firmware as a “no exception recorded” marker.

Looking ahead to testing, we will use these `mcause` outcomes as concrete pass/fail values when examining protection policies. For example, when execute permission is intentionally withheld from an unverified region, we expect `kIbexExcInstrAccessFault`, if execution is permitted over all-ones filler, we expect `kIbexExcIllegalInstrFault`. Likewise, read or write probes into protected areas should yield `kIbexExcLoadAccessFault` or `kIbexExcStoreAccessFault`. By comparing the observed to the expected `mcause` codes across our testing, we can validate secure boot assumptions, as well as analyse and learn from OpenTitan’s PMP/ePMP layouts.

6.3.1.6 Summary and motivation for ePMP PMP lets machine mode partition the physical address space into regions with explicit R/W/X permissions using `pmpaddr` and `pmpcfg`. Entries are matched in index order (the lowest index wins). In base PMP, two behaviors create risk if the configuration is incomplete or wrong: (i) if an access does not match any entry, it is allowed by default for S/U, and (ii) M-mode ignores PMP completely unless an entry’s `L` bit is set. In practice, this means that a small mistake such as leaving a gap between regions (TOR boundary off by one), failing to set the `L` bit or simply not configuring a memory region can open “holes” where code is able to execute from unintended areas or where M-mode is able to rewrite the protections. These pitfalls are common during early boot, when only a few entries are programmed and the system is still running in M-mode.

The enhanced PMP (ePMP) extension addresses exactly these pain points. It introduces the `mseccfg` CSR to control machine-mode lockdown and default policy, enabling a true whitelist-by-default behavior and consistent enforcement in M-mode during secure boot. In the next subsection (6.3.2) we describe how ePMP’s policy bits are used by the ROM to lock down execution, remove the default-allow gaps, and keep the configuration robust throughout the boot chain.

6.3.2 ePMP

6.3.2.1 Bridge and scope Building on the limitations outlined above, the enhanced PMP (ePMP, standardized as `Smepmp[11]`) introduces machine mode policy controls that enable whitelist by default behavior and reliable enforcement during secure boot, without permanently consuming extra PMP entries or relying on rule ordering.

6.3.2.2 The `mseccfg` CSR ePMP adds the machine-mode CSR `mseccfg` to control early-boot security policy. The register is accessible only in M-mode and defines policy bits that alter PMP’s default behavior while remaining fully compatible with base PMP when these bits are left cleared. On Ibex/OpenTitan only a subset is used and the remaining bits are reserved.

6.3.2.3 Rule Locking Bypass (`mseccfg.RLB`) This bit provides a controlled, temporary override of the locked

state so that boot code can safely evolve PMP after first constraining M-mode:

- **If `RLB=1`:** software may modify or remove entries even when `pmpcfg.L=1`, allowing the ROM to lock down early, verify the next stage, and then reconfigure cleanly.
- **If `RLB=0` and any entry has `L=1`:** `RLB` is held at 0 and further writes to set it are ignored until a PMP reset. This prevents late-stage code from reopening a bypass once protections are committed. This stickiness is a defense-in-depth choice to avoid compromising earlier security features.

6.3.2.4 Machine-Mode Whitelist Policy (`mseccfg.MMWP`) This bit changes the default outcome for M-mode when no PMP entry matches:

- **If `MMWP=1`:** unmatched accesses are denied (true whitelist by default).
- **If `MMWP=0`:** base PMP semantics apply, therefore, unmatched accesses are not blocked by PMP (subject to `L` semantics).

In practice, platforms enable `MMWP` early or wire it through RTL configuration and leave it on through the secure-boot chain to avoid reintroducing default-allow windows.

6.3.2.5 Machine-Mode Lockdown (`mseccfg.MML`) When enabled, `MML` reinterprets the meaning of the `pmpcfg.L` bit to partition enforcement by privilege domain. Concretely, if `pmpcfg.L == 1`, the corresponding PMP entry is enforced only in M-mode (and is not applied to U/S mode). If `pmpcfg.L == 0`, the entry is enforced only in U/S-mode (and is ignored in M-mode). When `MML == 0`, base PMP semantics apply instead: `L` locks the entry and extends its enforcement to M-mode (otherwise M-mode ignores the entry). OpenTitan’s standard flow does not rely on `MML` therefore we do not use it in our experiments. For the formal definition and additional edge-case behavior, refer to the `Smepmp` specification[11].

6.3.2.6 Why ePMP matters for OpenTitan During ROM and ROM_EXT, only a minimal verified ROM region should be executable, while all other regions should be non-executable. `MMWP` removes default-allow holes, while `RLB` gives the boot flow a safe way to lock early and still evolve the ePMP layout once the next stage is authenticated. This yields a more robust secure-boot posture than base PMP alone, while preserving compatibility (by leaving `mseccfg` cleared) for ablation tests and comparisons.

6.3.3 OpenTitan’s ePMP configuration

In this section we analyse how OpenTitan configures ePMP across reset and early boot, highlighting the transition from a minimal reset initialization to the fully-fledged execution capable configuration after verification of the next stage.

TABLE 2: ROM-STAGE ePMP ENTRIES

Entry	Description	Permissions	Addressing Mode
1	ROM TEXT	RX	TOR
2	ROM	R	NAPOT
5	eFlash	R	NAPOT
11	MMIO	RW	TOR
14	Stack Guard	<none>	NA4
15	RAM	RW	NAPOT

ROM-stage ePMP entries programmed by ROM. ROM_EXT remains non-executable until explicitly unlocked after signature verification

6.3.3.1 Hardwired Configuration At reset, Ibex loads a minimal ePMP state from RTL constants in *hw/top_earlgrey/rtl/ibex_pmp_reset_pkg.sv*. The arrays *PmpCfgRst[16]* and *PmpAddrRst[16]* predefine a small set of regions while leaving the remainder *OFF*. Concretely, entries cover **ROM**, **MMIO**, and **Debug ROM**, while other regions are intentionally left undefined. The machine security configuration CSR reset value sets *mseccfg.RLB=1* and *mseccfg.MMWP=1* with *mseccfg.MML=0*. Thus, unmatched addresses are denied by default in M-mode, and the ROM is allowed to refine or remove locked entries during early boot. Leaving unused entries *OFF* is therefore safe: with *MMWP=1* there are no default-allow gaps at reset.

[Maybe add Annex with RTL code?]

6.3.3.2 ROM’s ePMP Init Table 2 summarizes the ROM-stage ePMP layout: **ROM TEXT** is RX and bounded with *TOR* to tightly cover only instructions, **ROM** data/const is R via *NAPOT* and is non-executable, **eFlash** is R-only so the ROM can read the *ROM_EXT* manifest and image for verification, but *ROM_EXT* is not executable at this stage, **MMIO** is RW with *TOR*, the 4-byte **stack guard** is *NA4* with no access, and **RAM** is RW via *NAPOT* and never executable. With *MMWP=1*, any address not covered by an allow-entry is denied by default.

6.3.3.3 Allowing ROM_EXT’s execution Once the ROM has validated the *ROM_EXT*’s manifest and signature, it updates ePMP to permit execution of the *ROM_EXT* text only. Practically, the function *rom_epmp_unlock_rom_ext_rx* programs a *TOR* pair to bound **ROM_EXT TEXT** with RX and finally configures a *NAPOT* R region for **ROM_EXT** read-only sections. During this step *mseccfg.RLB=1* allows modification of entries that were previously locked. After hand-off, later stages can restrict reconfiguration by clearing/binding off that flexibility. This staged enablement ensures that flash becomes executable only after cryptographic verification, while the deny by default posture remains in effect for all unmatched regions.

6.4 Cryptographic Verification

[TBD]

6.5 Anti-rollback measures

[TBD]

[A partir d’aquí, WiP]

7 METHODOLOGY AND PLANNING

Regarding the methodology, we will make use of simulators such as Verilator, which transforms our digital design (Verilog code) of OpenTitan’s Earlgrey SoC into high-level C++ code that allows us to emulate our platform very efficiently, though not with complete accuracy.

OpenTitan provides several tools to carry out development with Verilator, which, depending on the execution flow, are the following:

1. Generating the simulator from the RTL

- FuseSoC** gathers all the RTL, auto-generates SystemVerilog files if needed, and passes the file list to Verilator.
- Verilator** translates the Verilog/SystemVerilog into C++ and compiles it, via *make*, to produce the simulation binary *Vchip_sim.tb*.

2. Building the software for the simulated SoC

- Bazel** builds all the software, including tests. It also produces the ROM, Flash, and OTP images that will be loaded into the simulation (SRAM is filled during runtime).

3. Launching the simulation

- Opentitantool** is invoked by Bazel and boots the Verilator binary with the appropriate arguments. It then connects to the simulation UART to display real-time output.

Once the simulation has started, it is possible to run any test in the repository using the *verilator* flag. There are also other categories of tests, such as those intended for *DV* (Design Verification), which use EDA (Electronic Design Automation) tools and fall outside the scope of this work, or those specifically designed for *FPGA*.

Regarding the main methodology of the project, we will work with two instances of the simulation or execution system, through the Verilator workflow. One instance will be used to run the baseline tests, and the other will be used for our experimental modifications, removing, relaxing, or altering certain security policies and observing the impact these changes have on the system’s overall security.

An emulation approach using Field-Programmable Gate Arrays (FPGAs) was initially considered, however, given schedule constraints and late hardware availability, we considered the approach beyond the scope of this thesis.

7.1 ePMP baseline testing

This section describes the test approach used to validate OpenTitan’s boot memory protection. The goal is to establish a reproducible baseline for ePMP behavior, then contrast it with base PMP, and finally study the security impact of selectively removing ePMP features. All tests

are driven from the ROM/ROM_EXT context and observe outcomes via architectural signals, primarily `mcause` and `mepc`. Probes cover three access types, *execute*, *load*, and *store*, against chosen addresses: (i) inside a region explicitly covered by a PMP entry, (ii) on region boundaries, and (iii) outside any defined region. Pass/fail criteria are the expected traps for disallowed operations and successful completion for allowed ones.

[Add Annex with execute code, and exception handler ?]

7.1.1 Access control for configured regions

We first validate the programmed permissions for regions explicitly covered by ePMP entries at ROM time (Table 2). For each region we issue *execute*, *load*, and *store* probes to interior addresses and to the first/last word of the interval. **ROM TEXT** accepts execution and loads (RX), rejects stores (expected `kIbexExcStoreAccessFault`), and so on.

7.1.2 Default-deny for unconfigured regions (MMWP=1)

Next, we probe addresses that aren't covered by any active entry. With `mseccfg.MMWP=1`, unmatched accesses in M-mode are denied by default. Execute probes at unmapped flash/RAM addresses raise `kIbexExcInstrAccessFault`, loads raise `kIbexExcLoadAccessFault`, and stores raise `kIbexExcStoreAccessFault`. These outcomes confirm that a catch-all deny rule is unnecessary under ePMP: the policy itself enforces a whitelist during early boot.

7.1.3 ROM_EXT transition: from read-only to executable

Before verification, **ROM_EXT** resides in flash as R-only and is non-executable. We plan two checkpoints: before and after unlock. Pre-unlock execute probes into ROM_EXT fault with `kIbexExcInstrAccessFault` while loads succeed (to feed the verification routine). After successful signature validation, the ROM programs a *TOR* pair to grant RX on the ROM_EXT text region. Post-unlock, the same execute probe completes without trap, while stores remain disallowed. This demonstrates that execution permission for the next stage is dependent on cryptographic verification and an explicit ePMP update.

7.2 Comparison: base PMP vs. ePMP

We repeat a minimal subset of the above probes with `mseccfg=0` (base PMP semantics). Within configured regions, behavior matching ePMP for the same R/W/X bits has been observed. The key difference appears on unconfigured addresses: under base PMP, unmatched accesses in M-mode are permitted by default, so execute/load/store probes complete unless some other entry happens to match. This duality of comparisons isolates the contribution of MMWP to the whitelist-by-default posture and shows why ePMP is preferable for early boot, when only a small set of entries is active.

7.3 Ablation: selective removal of ePMP features

We remove one feature at a time and re-run the probes to measure impact.

7.3.1 Early-boot exposure with MMWP=0

To quantify the risk window before ePMP is configured, we evaluate baseline PMP behavior when the machine-mode whitelist is disabled. Concretely, we (i) force `mseccfg.MMWP=0` at reset (or clear it before any PMP lock is taken), and (ii) comment out the first call to `rom_epmp_init`, leaving most PMP entries *OFF*. From the ROM context we place probe words (e.g., all ones `0xFFFF...`) at addresses outside any configured region, such as unused RAM, eFlash areas not yet covered by an entry, and gaps between tentative *TOR* bounds, and attempt to transfer control there, thus creating a substantial risk: M-mode is now allowed to execute from almost any region outside ROM, defeating the purpose of cryptographically verifying firmware stages, since jumping to unverified code now becomes plausible.

7.3.1.1 Expectation. With MMWP=0, base PMP semantics apply to M-mode: if an access does not match any PMP entry, it is permitted by default. Therefore, an *execute* probe to an unmapped address should *not* fault on access-control grounds. When the probe word is not a valid opcode, the expected trap is `kIbexExcIllegalInstrFault`; if valid code is present, execution proceeds. By contrast, with MMWP=1 the same probe must raise `kIbexExcInstrAccessFault` because unmatched regions are denied in M-mode.

7.3.1.2 Result. With MMWP=0 and no ePMP init we observe no access-control trap when jumping to unmapped locations; the observed `mcause` is `kIbexExcIllegalInstrFault` for the all-ones payload, confirming that control reached the target and only failed at decode. The same probes with MMWP=1 raise `kIbexExcInstrAccessFault`, demonstrating whitelist-by-default behavior.

7.3.1.3 Implication. If MMWP=0 during early boot and the ROM defers ePMP initialization, M-mode can execute from regions not explicitly denied, creating an opportunity for redirection before protections are established. This motivates (i) hardwiring MMWP=1 at reset, (ii) keeping only a minimal RX window around ROM text (via *TOR*), and (iii) invoking ePMP setup as an early ROM action so that unmatched regions are non-executable from the outset.

7.4 Cryptographic Firmware Verification

[TBD]

8 RESULTS CONCLUSION

ACKNOWLEDGEMENTS

[TBD]

REFERENCES

- [1] *OpenTitan: Open Source Silicon Root of Trust*, Disponible: <https://opentitan.org>
- [2] A. Meza, C. Tunc, P. Nasahl, and J. Grossklags, “Security Verification of the OpenTitan Hardware Root of Trust,” doi: 10.1109/MSEC.2023.3255363.
- [3] A. Musa, E. Parisi, L. Barbierato, A. Acquaviva, and F. Barchi, “End-to-end Integration of OpenTitan Security Features in a Pure RISC-V SoC,” doi: 10.1109/SMACD61181.2024.10745397.
- [4] S. K. Saha, A. N. Butka, M. K. Ahmed, and C. Bobda, “OpenTitan based Multi-Level Security in FPGA System-on-Chips,” doi: 10.1109/ICFPT59805.2023.00056.
- [5] *Secure Memory Protection Extension (PDF)*, Github: riscvarchive/riscv-tee Project. Disponible: <https://github.com/riscvarchive/riscv-tee/blob/191b563b08b31cc2974d604a3b670d8666a2e093/Smepmp/Smepmp.pdf> (Accedit: 2025-09-23)
- [6] *The RISC-V Instruction Set Manual (Unprivileged ISA)*. RISC-V International. Disponible: <https://riscv.org/technical/specifications/> (Accedit: 2025-11-15)
- [7] *Hardware Root of Trust*. Rambus Blog. Disponible: <https://www.rambus.com/blogs/hardware-root-of-trust/> (Accedit: 2025-11-15)
Keystone: An Open Framework for Architecting Trusted Execution Environments. EuroSys 2020. Disponible: <https://www.shwetashinde.org/publications/keystone-eurosys20.pdf> (Accedit: 2025-11-15)
- [8] *RISC-V Memory Protection: Diving deep into the complexities*. Medium (Karthik B K). Disponible: <https://medium.com/@talktokarthikbk/risc-v-memory-protection-diving-deep-into-the-complexities-9d751212b6b8> (Accedit: 2025-11-15)
- [9] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture (inclou PMP)*. RISC-V International. Disponible: <https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf> (Accedit: 2025-11-15)
- [10] *Verified Boot Crypto*. Chromium OS Design Docs. Disponible: [https://www.chromium.org/chromiumos-design-docs/verified-boot-crypto/](https://www.chromium.org/chromium-os/chromiumos-design-docs/verified-boot-crypto/) (Accedit: 2025-11-15)
- [11] *Secure Memory Protection Extension (PDF)*. GitHub: riscvarchive/riscv-tee Project. Disponible: <https://github.com/riscvarchive/riscv-tee/blob/191b563b08b31cc2974d604a3b670d8666a2e093/Smepmp/Smepmp.pdf> (Accedit: 2025-11-15)
- [12] *OpenTitan: Open source silicon root of trust (RoT)*. Project homepage. Disponible: <https://opentitan.org/> (Accedit: 2025-11-15)
- [13] *OpenTitan Partnership Makes History: First Open-Source Silicon to Reach Commercial Availability*. lowRISC News (13 Feb 2024). Disponible: <https://lowrisc.org/news/opentitan-commercial-availability/> (Accedit: 2025-10-04)
- [14] “A booting computer is as vulnerable as a newborn baby”. ETH Zürich News (5 Nov 2019). Disponible: <https://ethz.ch/en/news-and-events/eth-news/news/2019/11/project-opentitan.html> (Accedit: 2025-11-15)
- [15] *Nuvoton Develops OpenTitan®-based Security Chip as Commercial Product*. Press Release (30 May 2024). Disponible: <https://www.nuvoton.com/news/news/all/TSNuvotonNews-000514/> (Accedit: 2025-11-15)
- [16] *Fabrication begins for production OpenTitan silicon*. Google Open Source Blog (6 Feb 2025). Disponible: <https://opensource.googleblog.com/2025/02/fabrication-begins-for-production-opentitan-s.html> (Accedit: 2025-11-15)
- [17] *Seagate Designs RISC-V Cores to Power Data Mobility and Trustworthiness*. Seagate Article. Disponible: <https://www.seagate.com/stories/articles/seagate-designs-risc-v-cores-to-power-data-mobility-and-trustworthiness> (Accedit: 2025-11-15)
- [18] *Western Digital Collaborates with lowRISC and Google to Increase Security Transparency in Data-Centric Platforms*. Western Digital Press Release (5 Nov 2019). Disponible: <https://www.westerndigital.com/company/newsroom/press-releases/2019/2019-11-05-western-digital-collaborates-with-lowrisc-and-google-to-increase-security-transparency-in-data-centric-platforms> (Accedit: 2025-11-15)
- [19] *OpenTitan production silicon and early integrations*. lowRISC News (20 Feb 2025). Disponible: <https://lowrisc.org/news/the-worlds-first-open-source-security-chip-hits-production/> (Accedit: 2025-11-15)
- [20] *Hardware Trojans in Chips: A Survey for Detection and Prevention*. Sensors (MDPI), vol. 20, no. 18, 5165 (2020). Disponible: <https://www.mdpi.com/1424-8220/20/18/5165> (Accedit: 2025-11-15)
- [21] *OpenTitan Lightweight Threat Model*. Project documentation. Disponible: <https://opentitan.org/>

org/book/doc/security/threat_model/
index.html (Accedit: 2025-11-15)

- [22] *Design Approaches and Architectures of RISC-V SoCs (referència a Sv32)*. RISC-V International Blog. Disponible: <https://riscv.org/blog/design-approaches-and-architectures-of-risc-v-socs/> (Accedit: 2025-11-15)

APPENDIX